2025 International Annual Conference on Complex Systems and Intelligent Science (CSIS-IAC)
May 16~18, 2025, Shenzhen, China

Cluster-LLM: Adaptive Real-Time Time-Series Anomaly Detection Using LLMs

Binbin Zhu[1,2*], Gang Xiong[1*], Meng Yuan[1,2], Zhen Shen[1], Fenghua Zhu[1], Shichao Chen[1], Xisong Dong[1], Sheng Liu[1†], Junrui Chen[1,2]
[1]State Key Laboratory of Multimodal Artificial Intelligence Systems,
Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China
[2]School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China
Email: zhubinbin2023@ia.ac.cn, gang.xiong@ia.ac.cn, yuanmeng2022@ia.ac.cn,
zhen.shen@ia.ac.cn, fenghua.zhu@ia.ac.cn, shichao.chen@ia.ac.cn,
xisong.dong@ia.ac.cn, sheng.liu@ia.ac.cn, chenjunrui2022@ia.ac.cn

2025 International Annual Conference on Complex Systems and Intelligent Science (CSIS-IAC) | 979-8-3315-9768-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/CSIS-IAC65538.2025.11161558

***Abstract*—Time-series anomaly detection plays a vital role in industrial monitoring, finance, and cybersecurity. However, existing methods often suffer from limited labeled data, poor generalization, and high detection latency. In this paper, we propose Cluster-LLM, an unsupervised anomaly detection method that combines time-series clustering with large language models (LLMs). The method captures temporal pattern shifts through clustering and leverages chain-of-thought (CoT) prompting to guide the LLM in inferring adaptive baselines. A confidence-weighted mechanism is introduced to enable real-time detection.Experiments conducted on the KPI and Yahoo benchmark datasets demonstrate that Cluster-LLM achieves a precision of 0.985 and 0.885 respectively, while reducing detection latency to 283s and 23s, substantially outperforming several unsupervised baselines. In addition, it maintains competitive F1-scores of 0.642 (KPI) and 0.537 (Yahoo) despite being entirely unsupervised. These results highlight the potential of LLMs in real-time time-series anomaly detection.**

***Index Terms*—Time-Series Anomaly Detection, Large Language Models, Chain-of-Thought Prompting, Real-Time Detection**

I. Introduction

Time-series anomaly detection is a crucial technique for identifying abnormal events within normal time-series data and is widely applied in domains such as industrial equipment maintenance, network security, and financial transactions. However, in real-world industrial environments, high-quality labeled datasets are scarce, and the cost of manual labeling is prohibitively high. In addition, unexpected downtime of critical equipment can result in substantial economic losses, which makes timely detection and response to anomalies essential.

Existing time series anomaly detection methods often face challenges such as heavy reliance on labeled data, poor timeliness, and insufficient generalization performance. To address these issues, this paper proposes an innovative unsupervised time series anomaly detection method that combines clustering techniques with LLMs. The proposed approach first uses clustering to capture pattern shifts in time series data and then uses the powerful modeling capabilities of LLMs to identify anomalies, enabling fast and accurate anomaly detection. Compared to traditional methods, Cluster-LLMs demonstrate exceptional performance in real-time detection scenarios, even in an unsupervised setting.

The contributions of this paper are highlighted below.

- The purpose of this paper is to propose a low-latency time series anomaly detection method that maintains high accuracy, making it highly suitable for latency-sensitive detection scenarios.
- Introducing a dynamic baseline determination approach that adapts to environmental changes by analyzing multiple time dimensions, effectively reducing the false alarm rate.
- Innovatively applying LLMs to time-series anomaly detection, showcasing their potential in capturing complex temporal patterns and improving anomaly detection performance.

The remainder of this paper is organized as follows. First, Section 2 provides an overview of existing time series anomaly detection methods. Then, we introduce our methodology in Section 3 and discuss the experimental setup and performance evaluation in Section 4. Finally, Section 5 concludes the paper and outlines directions for future research.

II. Related Work

Time series anomaly detection techniques can be broadly categorized into four main approaches: statistical methods, machine learning methods, deep learning methods, and large model methods. Each approach offers unique advantages and challenges, with varying levels of complexity and assumptions about the data.

Statistical methods primarily rely on probability distributions and statistical assumptions to detect anomalies. These approaches generally assume that normal data follows a known distribution, such as a normal or Poisson distribution, and

* The work was supported by Beijing Natural Science Foundation under grant L233005 and L243009, National Natural Science Foundation of China under Grant U24A20277, the State Key Laboratory of Industrial Control Technology, China (No. ICT2024B10). Binbin Zhu and Gang Xiong are co-first authors; Sheng Liu is Corresponding author.

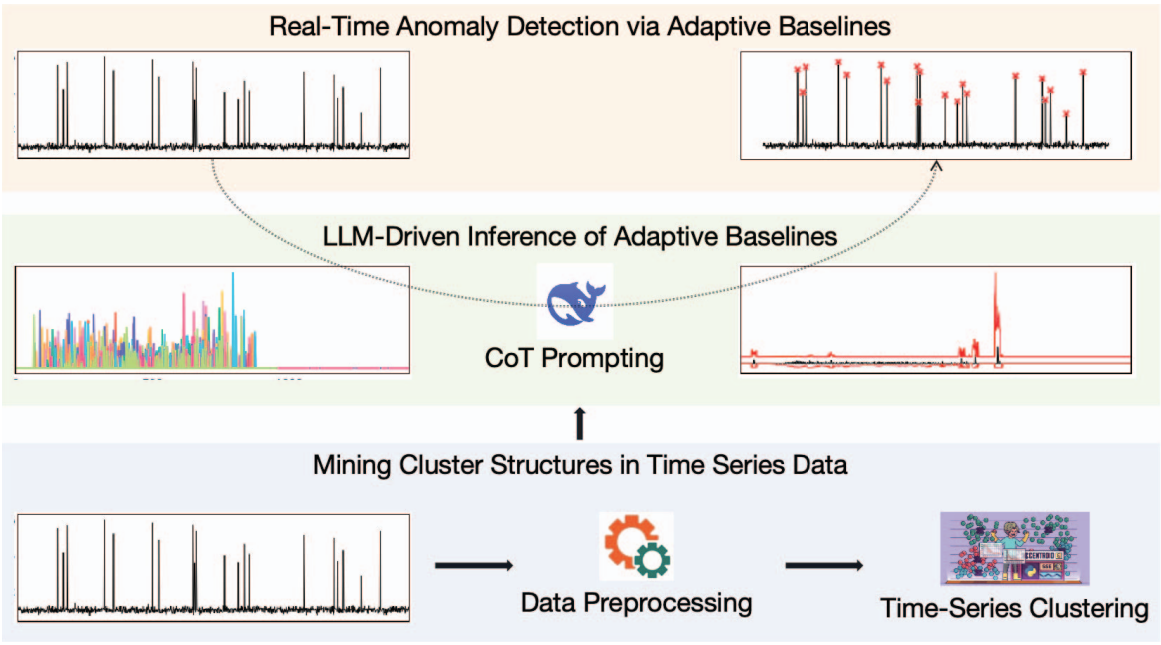


Fig. 1: Cluster-LLM Method Overview.

identify anomalies based on the degree of deviation from the expected distribution [1], [2].The z-score is a classical statistical method, where the core idea is to determine anomalies by calculating the deviation of a data point from the overall sample mean. The formula is as follows:

$$Z = \frac{X - \mu}{\sigma} \tag{1}$$

Here, X represents the sample value, μ represents the overall sample mean, and σ represents the standard deviation of the overall sample. Typically, if $|Z| > 3$, the data point is considered an anomaly; otherwise, it is considered normal.Autoregressive Integrated Moving Average (ARIMA) is a time-series forecasting model that is also widely used in the field of time-series anomaly detection. This method determines whether an observation is anomalous based on the residuals.

Statistical methods place very strict requirements on the probability distribution of the data, and in some scenarios, they may not achieve satisfactory results. With the advancement of machine learning technologies, methods such as K-Nearest Neighbors (KNN) and Support Vector Machine (SVM) have gradually been applied to the field of time-series anomaly detection. These methods typically have less stringent requirements on the data distribution [3], [4]. Li introduced a time-series anomaly detection algorithm that leverages the Local Outlier Factor (LOF) to effectively identify anomalies. This method employs hierarchical segmentation to extract key features and utilizes Dynamic Time Warping (DTW) to calculate the LOF [5].Marteau et al. proposed the Hybrid Isolation Forest (HIF) algorithm to enhance anomaly detection by addressing the ”blind spot” issue inherent in traditional Isolation Forests and by incorporating supervised learning capabilities [6]. Paffenroth et al. introduced a robust Principal Component Analysis (RPCA)-based method for anomaly detection in cyber networks. The proposed method optimizes the parameter λ to significantly enhance the accuracy of detecting network attacks [7].

Machine learning methods for time-series anomaly detection often rely on feature engineering. Deep learning methods, on the other hand, can capture temporal dependencies in time-series data while automatically extracting deeper features, and have been widely applied in the field of time-series anomaly detection. Deep learning methods can be broadly categorized into two types: prediction-based and reconstruction-based [8]. P. Malhotra et al. developed a time-series anomaly detection method using Recurrent Neural Networks (RNNs). By modeling the temporal dependencies within time series data, their approach effectively predicts future data points and identifies anomalous behavior based on the prediction error [9]–[11]. The reconstruction-based time-series anomaly detection method constructs a model to learn the normal patterns of the time-series, and then identifies anomalies based on the reconstruction error. For example, H. Xu et al. proposed a method for time-series anomaly detection based on Variational Autoencoders (VAE), which leverages the probabilistic framework of VAEs to effectively capture the underlying distribution of the data and detect anomalies through the reconstruction error [12]–[14]. Ren et al. proposed an innovative time-series anomaly detection method that integrates Spectral Residual (SR) with Convolutional Neural Network (CNN). This approach leverages the unique ability of SR to capture salient features in the frequency domain and combines it with the powerful pattern recognition capabilities of CNN, achieving state-of-the-art performance on multiple datasets [15].

Due to their high training costs and limited transferability,

deep learning models typically cannot be directly utilized across different scenarios without additional adaptation. Recent work suggests that pretrained LLMs offer noteworthy few/zero-shot capabilities and transferability [16], [17]. LLMs have shown remarkable capabilities in the field of natural language processing, and there is a growing trend in research to apply LLMs to other domains [18], [19]. A. Russell-Gilbert and colleagues introduced an innovative approach for anomaly detection in time series data, leveraging LLMs. This method, dubbed Adaptive Anomaly Detection Using Large Language Models (AAD-LLM), has been validated for its efficacy on datasets pertinent to the Predictive Maintenance (PdM) sector [20].

III. Methodology

This section presents the overall design of our proposed method, **Cluster-LLM**, which aims to achieve efficient and accurate time-series anomaly detection under an unsupervised setting. The method comprises four main stages: data preprocessing to handle missing values and noise while enriching temporal granularity; unsupervised clustering of time-series segments to identify representative temporal patterns; LLM-driven inference of adaptive upper and lower baselines using CoT prompting; and real-time anomaly detection via confidence-weighted scoring. An overview of the proposed framework is illustrated in Fig. 1, which depicts the end-to-end process from offline pattern modeling to online anomaly detection, highlighting the core modules and data flow of Cluster-LLM. The following subsections provide a detailed explanation of each component.

A. Data Preprocessing

Time-series is defined as a sequence of data points arranged in temporal order, consisting of timestamps and their corresponding values. The timestamp records the specific time information for each data point, while the value reflects the actual measurement obtained by the sensor at that particular moment. Table I illustrates the raw time series data collected by a certain sensor.

Time series data acquired from sensors is often subject to various influences, including network-related issues, which can result in missing values and noisy data.

TABLE I: Raw time series data collected by a sensor.

timestamp	value
1606760000000	15.6
1606820000000	23.4
1606880000000	23.2

TABLE II: Preprocessed time series data with detailed timestamp information.

year	month	day	hour	minute	value
2023	7	13	13	25	15.6
2023	7	13	13	26	23.4
2023	7	13	13	27	23.2

To address these challenges, we must initially preprocess the raw time series data. In cases of missing values, we commonly opt for imputation or removal; whereas, for noisy data, we generally employ smoothing techniques [7]. Subsequently, to facilitate clustering of the raw data at a daily granularity, we enhance the data by extracting detailed date and time information from the timestamps. Table II presents the data following this preprocessing phase.

B. Time Series Clustering

Clustering is an unsupervised learning technique designed to partition a dataset into multiple clusters, ensuring that data points within the same cluster are as similar as possible, while those in different clusters are as dissimilar as possible [21]. After evaluating several common clustering algorithms, including k-means, hierarchical clustering, and Density-Based Spatial Clustering of Applications with Noise (DBSCAN), we conducted extensive experiments and ultimately determined that the k-means algorithm was the most suitable for our specific application.

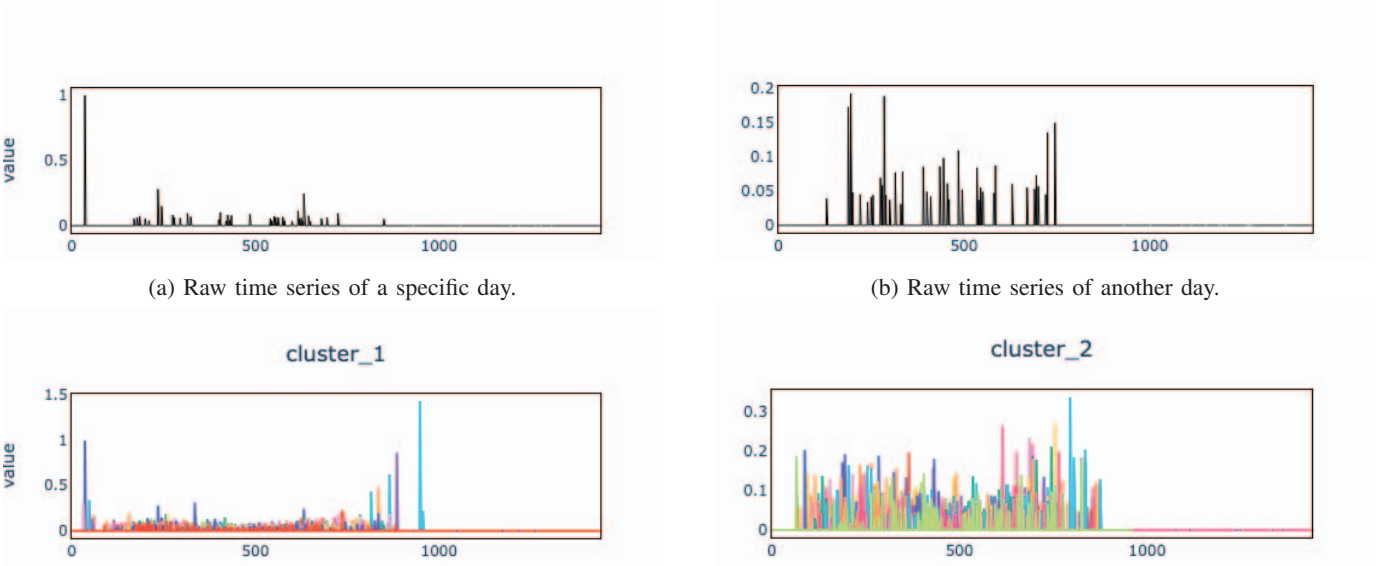
Algorithm 1 illustrates the basic principle of K-means.The pivotal parameter for clustering performance is k, the number of clusters. To ascertain the optimal k, we employ the Calinski-Harabasz index (CH), a metric that evaluates clustering effectiveness by assessing the variance between and within clusters, A higher index value indicates superior clustering quality [22]. Specifically, the (CH) is calculated using the following formula:

$$CH = \frac{tr(B_k)/(k-1)}{tr(W_k)/(N-k)} \tag{2}$$

where $tr(B_k)$ represents the between-cluster variance, and $tr(W_k)$ signifies the within-cluster variance, with N being the total number of data points and k denoting the number of clusters. This index provides a quantitative measure of clustering quality, facilitating the identification of the most appropriate k for our dataset.

Algorithm 1 k-means Clustering Algorithm

Require: Number of clusters k, set of data points $X = \{x_1, x_2, \ldots, x_n\}$
Ensure: Clusters $C = \{C_1, C_2, \ldots, C_k\}$
Initialize k cluster centroids $M = \{m_1, m_2, \ldots, m_k\}$ randomly from X
repeat
  for each data point x_i in X **do**
    Assign x_i to the nearest centroid m_j, where $j = \arg\min_{l=1}^{k} \text{distance}(x_i, m_l)$
    Form clusters $C_j = \{x_i | x_i \text{ is assigned to } m_j\}$
  end for
  for each cluster C_j **do**
    Update the centroid m_j to be the mean of all points in C_j
  end for
until convergence or maximum iterations reached



(a) Raw time series of a specific day.

(b) Raw time series of another day.

(c) First cluster of raw time series.

(d) The other cluster of raw time series.

Fig. 2: Raw time series samples and clustered clusters.

As depicted in Table II, the time series data have undergone preprocessing. To enhance the clustering performance, the preprocessed time series were segmented into three distinct time intervals: midnight to 8:00 AM, 8:00 AM to 4:00 PM, and 4:00 PM to midnight. This partitioning strategy is pivotal for capturing temporal patterns and variations within the data, thereby facilitating a more nuanced clustering analysis. Subsequently, feature extraction was performed on each time window, encompassing a range of metrics including, but not limited to, mean, variance, and entropy.

$$\begin{pmatrix} avg_1^{(1)} & std_1^{(1)} & avg_1^{(2)} & std_1^{(2)} & avg_1^{(3)} & std_1^{(3)} \\ avg_2^{(1)} & std_2^{(1)} & avg_2^{(2)} & std_2^{(2)} & avg_2^{(3)} & std_2^{(3)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ avg_n^{(1)} & std_n^{(1)} & avg_n^{(2)} & std_n^{(2)} & avg_n^{(3)} & std_n^{(3)} \end{pmatrix} \tag{3}$$

The matrix in Equation 3 represents the dataset used for training the clustering model. Each row corresponds to a distinct day, and each column corresponds to a feature extracted from the time series data within a specific window on that day. Specifically, $avg_i^{(j)}$ denotes the mean of the data within the j-th window on the i-th day, while $std_i^{(j)}$ represents the variance of the data within the same window. These features capture the central tendency and dispersion of the data within each window, providing a comprehensive representation of the daily patterns for clustering analysis. Ultimately, we utilized the dataset formulated in Equation 3, coupled with our approach for identifying the optimal k-value, to train our clustering model effectively.

As demonstrated in Fig. 2a and Fig. 2b, these subplots showcase the raw time series data collected by a sensor over two separate days. Following the clustering process, Fig. 2c and Fig. 2d illustrate the two distinct clusters that have been identified.

C. Adaptive Baselines via LLMs

Following the clustering of time series data, an essential next step involves establishing the upper and lower baselines for each cluster, together with a confidence score that quantifies the reliability of each baseline.

LLMs exhibit strong zero-shot and few-shot generalization capabilities. Among various prompting strategies, CoT prompting is widely used to enhance reasoning by emulating human cognitive processes—breaking down complex tasks into a sequence of interpretable sub-steps [23]. In this work, we leverage CoT prompting to guide the LLM in inferring the upper and lower baselines for each cluster. For each cluster, we decompose the baseline inference task into three interpretable sub-steps, as illustrated in Fig. 3. First, we compute the upper and lower baselines based on daily z-score statistics. Next, we apply a $\pm$5-minute sliding window centered on each minute to fit the local time series to a Gaussian distribution and derive the corresponding baselines. Finally, we fuse the results from the previous two steps to generate the final adaptive upper and lower baselines. Fig. 4 shows a sample result of the proposed baseline inference process for one cluster. The black curve represents the fitted time series values within the cluster, while the red and blue lines denote the inferred upper and lower baselines, respectively.

To improve the robustness of the baseline estimation, we assign a confidence score to each cluster based on its historical frequency. Specifically, the confidence score C_i for cluster i is defined as:

$$C_i = \frac{N_i}{N} \tag{4}$$

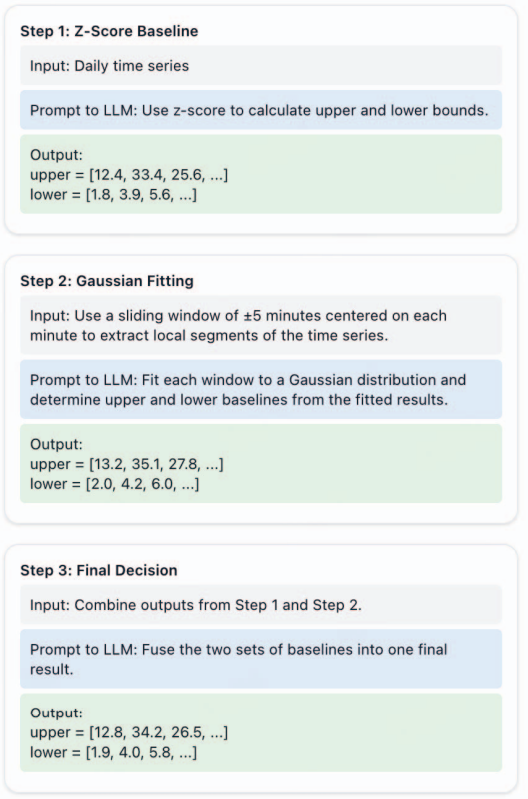


Fig. 3: CoT prompting pipeline for adaptive baseline estimation.

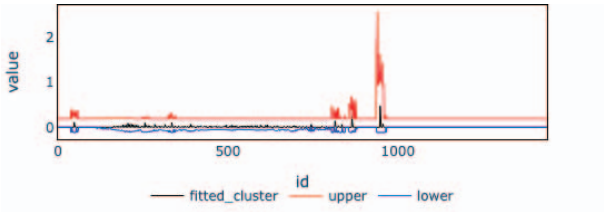


Fig. 4: Adaptive baseline bounds estimated for a sample cluster.

where N_i denotes the number of historical days assigned to cluster i, and N is the total number of historical days. This score reflects how frequently a given cluster pattern occurs in the past, serving as a measure of its reliability.

To support downstream real-time anomaly detection, we persist the essential metadata of each cluster, including the inferred upper and lower baselines and their associated confidence scores, into a structured database for long-term use and efficient access.

D. Real-Time Anomaly Detection

With the adaptive baselines and cluster-specific confidence scores established in the previous stage, we now introduce our real-time anomaly detection method. The goal is to monitor incoming time series data and detect deviations from normal patterns defined by the historical baselines. Algorithm 2

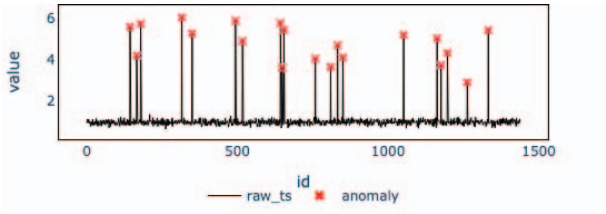


Fig. 5: Final detection results on a sample time series: the black curve represents the raw data, and red x markers indicate detected anomalies.

presents the real-time anomaly detection procedure. For each incoming data point, the system iterates through all historical clustering patterns and retrieves the corresponding upper and lower baselines along with their confidence scores. If the data point falls outside the inferred baselines, the confidence score of that cluster is accumulated into a total anomaly score. Once the loop completes, the total score is compared against a predefined threshold. If the score exceeds the threshold, the data point is flagged as anomalous. Otherwise, it is treated as normal. This design allows the detector to consider multiple potential patterns and their likelihoods, thereby improving sensitivity and reducing false positives in real-time settings.

Algorithm 2 Real-Time Anomaly Detection Procedure

Require: Real-time data to be inspected
Ensure: Anomaly detection result
Initialize;
$score \leftarrow 0.0$;
for each *pattern* in *patterns* **do**
  Retrieve upper/lower/confidence of the current cluster;
  if Data $<$ lower **or** Data $>$ upper **then**
    $score \leftarrow score + confidence$;
  end if
end for
if $score > threshold$ **then**
  Data is anomalous;
else
  Data is normal;
end if

To intuitively demonstrate the effectiveness of the proposed detection method, Fig. 5 visualizes a segment of the time series alongside its inferred baselines and detected anomalies. The black line represents the raw time-series values, while the red x-shaped markers highlight the points identified as anomalies.

IV. EXPERIMENTS AND DISCUSSIONS

A. Datasets

We conduct experiments on two publicly available benchmark datasets: the Yahoo anomaly detection dataset and the KPI dataset released by the AIOps competition.The statistics of these datasets are shown in Table III.

TABLE III: Statistics of datasets

DataSet	Real/Synth	Entities	Total Points	Anomaly Points
KPI	Real	29	5,922,913	134,114 / 2.26%
Yahoo	Real/Synth	367	572,966	3,896 / 0.68%

Yahoo is an open dataset for anomaly detection released by Yahoo Labs. It consists of both real and synthetic time-series, each labeled with anomaly points. The dataset is designed to evaluate the accuracy of detecting different types of anomalies, such as point outliers and change-points. The synthetic subset includes series with diverse trends, seasonality, and noise levels, while the real subset contains metrics from Yahoo's online services, reflecting real-world behaviors and anomalies.

KPI is a dataset provided by the AIOps data competition, containing multiple key performance indicator (KPI) time series with anomaly annotations. These data were collected from various Internet companies, including Sogou, Tencent, and eBay. Most KPI curves have a 1-minute sampling interval, while a few series have a 5-minute interval [15].

Both datasets cover a wide range of anomaly types and temporal patterns, making them suitable for evaluating the generalization ability and low-latency performance of the proposed method.

B. Evaluation Metrics

To evaluate the performance of our proposed anomaly detection method, we adopt several standard metrics widely used in the anomaly detection domain, including precision, recall, and F1-score.

Precision measures the proportion of correctly identified anomalies among all detected anomalies, and is defined as:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5}$$

Recall measures the proportion of correctly detected anomalies among all actual anomalies, and is defined as:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{6}$$

F1-score is the harmonic mean of precision and recall, providing a balanced assessment when both are important:

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

In the above equations:

- TP (True Positives): Number of correctly detected anomaly points.
- FP (False Positives): Number of normal points incorrectly detected as anomalies.
- FN (False Negatives): Number of true anomaly points that are missed.

In addition, to evaluate the responsiveness of the proposed method, we measure the **detection latency**, defined as the computational time required to complete a single instance of anomaly detection. Lower latency indicates better runtime efficiency and suitability for real-time applications.

C. Experimental Results

This section presents the quantitative experimental results of the proposed Cluster-LLM method on the KPI and Yahoo datasets. We compare our approach against several baseline methods, all of which are unsupervised, using standard evaluation metrics including Precision, Recall, F1-score, and Detection Latency. To infer dynamic baselines, we utilize the `deepseek-r1:8b` model—an open-source large language model released by DeepSeek, containing 8 billion parameters. The model demonstrates strong zero-shot and few-shot reasoning capabilities, making it particularly suitable for our multi-step prompting strategy based on chain-of-thought (CoT) reasoning [23].

We evaluate Cluster-LLM alongside four representative unsupervised baseline methods: LOF, LSTM-VAE, DONUT and SR-CNN. These methods cover a diverse range of anomaly detection paradigms, including density-based approaches, deep generative models, and sequence modeling techniques.

As shown in Table IV, Cluster-LLM achieves highly competitive performance across both datasets. Although SR-CNN attains the highest F1-score, our method surpasses all baselines in precision, achieving 0.985 on KPI and 0.885 on Yahoo, which indicates a significantly lower false positive rate. This is particularly important in real-world scenarios where false alarms can lead to unnecessary costs or disruptions.

In terms of recall, LSTM-VAE and DONUT exhibit relatively higher values, especially on KPI and Yahoo respectively. However, they come at the cost of notably increased runtime 72,500 seconds for LSTM-VAE and 24,248 seconds for DONUT on KPI, making them less suitable for real-time or resource-constrained environments.

Cluster-LLM strikes a favorable balance between detection accuracy and efficiency. It completes inference within 283 seconds on KPI and just 23 seconds on Yahoo, significantly outperforming other baselines in latency. This demonstrates the effectiveness of using LLM-driven adaptive baselines for efficient anomaly detection, without relying on heavy training or end-to-end neural architectures.

These results collectively validate that LLMs, when equipped with appropriate prompting strategies such as chain-of-thought, are highly promising tools for time-series anomaly detection. They offer a novel and flexible paradigm for modeling temporal dynamics and reasoning under uncertainty, beyond traditional model-driven or data-driven approaches.

V. CONCLUSION

In this paper, we proposed Cluster-LLM, a novel method for time-series anomaly detection that combines unsupervised clustering with LLMs for adaptive baseline inference. By employing CoT prompting, our approach decomposes baseline estimation into interpretable reasoning steps, enabling LLMs to better understand temporal structures and statistical characteristics within each cluster. A confidence-weighted mechanism is further introduced to support robust real-time anomaly detection.

TABLE IV: Performance comparison of different methods on KPI and Yahoo datasets.

Method	KPI				Yahoo			
	F_1-score	Precision	Recall	Time(s)	F_1-score	Precision	Recall	Time(s)
LOF	0.388	0.342	0.451	3200	0.230	0.213	0.252	45
LSTM-VAE	0.737	0.674	**0.813**	72500	0.611	0.758	0.512	6907
DONUT	0.347	0.371	0.326	24248	0.026	0.013	**0.825**	2572
SR-CNN	**0.771**	0.797	0.747	2724	**0.652**	0.816	0.542	125
Cluster-LLM (Ours)	0.642	**0.985**	0.477	**283**	0.537	**0.885**	0.389	**23**

We evaluated Cluster-LLM on two widely used benchmark datasets: KPI and Yahoo. Compared with several representative unsupervised baselines, our method consistently achieved superior precision and the lowest detection latency, while also reaching near-optimal F1-scores. These results demonstrate that LLMs, when guided by carefully designed prompts, hold significant potential for time-series anomaly detection tasks.

Recent advances in LLM fine-tuning have shown that task-specific adaptation can significantly enhance model performance, especially in domain-sensitive applications such as code generation, scientific QA, and structured reasoning. Fine-tuning allows LLMs to internalize domain knowledge and align their output behavior with specific downstream tasks, reducing reliance on complex prompt engineering while improving consistency and accuracy.

In future work, we aim to explore fine-tuning LLMs specifically for time-series anomaly detection. By training the model on task-relevant temporal data and labeled anomaly patterns, we expect to improve the model's capability to infer context-aware baselines and detect nuanced deviations.

REFERENCES

[1] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, "Anomaly detection in streams with extreme value theory," in Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, pp. 1067–1075, Aug. 2017.

[2] A. Mahimkar, Z. Ge, J. Wang, J. Yates, Y. Zhang, J. Emmons, B. Huntley, and M. Stockert, "Rapid detection of maintenance-induced changes in service performance," in Proc. 7th ACM Conf. Emerg. Netw. Exp. Technol. (CoNEXT), pp. 13, Dec. 2011.

[3] D. Liu, Y. Zhao, H. Xu, Y. Sun, D. Pei, J. Luo, X. Jing, and M. Feng, "Opprentice: Towards practical and automatic anomaly detection through machine learning," in Proc. 2015 Internet Measurement Conf., pp. 211–224, Nov. 2015.

[4] D. Shipmon, J. Gurevitch, P. M. Piselli, and S. Edwards, "Time series anomaly detection: Detection of anomalous drops with limited features and sparse examples in noisy periodic data," Tech. Rep., Google Inc., 2017.

[5] Z. Li, Z. Li, N. Yu, S. Wen, et al., "Locality-based visual outlier detection algorithm for time series," Security and Communication Networks, vol. 2017, 2017.

[6] P. Boniol, Q. Liu, M. Huang, T. Palpanas, and J. Paparrizos, "Dive into time-series anomaly detection: A decade review," unpublished, Dec. 2024. doi: 10.48550/arXiv.2412.20512.

[7] R. Paffenroth, K. Kay, and L. Servi, "Robust PCA for anomaly detection in cyber networks," arXiv preprint arXiv:1801.01571, 2018. [Online]. Available: http://arxiv.org/abs/1801.01571

[8] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep learning for time series anomaly detection: a survey," ACM Comput. Surv., vol. 57, no. 1, pp. 1–42, Jan. 2025, doi: 10.1145/3691338.

[9] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long short term memory networks for anomaly detection in time series," in Proc. 2015 Int. Conf. Computational Intelligence, 2015.

[10] L. Bontemps, V. L. Cao, J. McDermott, and N.-A. Le-Khac, "Collective anomaly detection based on long short-term memory recurrent neural networks," in Proc. Int. Conf. Future Data Secur. Eng., pp. 141–152, 2016.

[11] F. Liu, X. Zhou, J. Cao, Z. Wang, T. Wang, H. Wang, and Y. Zhang, "Anomaly detection in quasi-periodic time series based on automatic data segmentation and attentional LSTM-CNN," IEEE Trans. Knowl. Data Eng., vol. 34, no. 6, pp. 2626–2640, Jun. 2022, doi: 10.1109/TKDE.2020.3014806.

[12] H. Xu et al., "Unsupervised anomaly detection via Variational Auto-Encoder for seasonal KPIs in web applications," in Proc. 2018 World Wide Web Conf. (WWW '18), pp. 187–196, 2018, doi: 10.1145/3178876.3185996.

[13] W. Chen et al., "Unsupervised anomaly detection for intricate KPIs via adversarial training of VAE," in Proc. IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, Paris, France, pp. 1891–1899, Apr. 2019, doi: 10.1109/INFOCOM.2019.8737430.

[14] Z. Li, W. Chen, and D. Pei, "Robust and unsupervised KPI anomaly detection based on conditional variational autoencoder," in Proc. IEEE 37th Int. Performance Computing Commun. Conf. (IPCCC), Orlando, FL, USA, pp. 1–9, Nov. 2018, doi: 10.1109/PCCC.2018.8710885.

[15] H. Ren et al., "Time-Series anomaly detection service at Microsoft," in Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, pp. 3009–3017, Jul. 2019, doi: 10.1145/3292500.3330680.

[16] X. Liu, D. J. McDuff, G. Kovács, I. R. Galatzer-Levy, J. Sunshine, J. Zhan, M.-Z. Poh, S. Liao, P. D. Achille, and S. N. Patel, "Large language models are few-shot health learners," arXiv preprint arXiv:2305.15525, 2023.

[17] Z. Chen, M. M. Balan, and K. Brown, "Language models are few-shot learners for prognostic prediction," arXiv preprint arXiv:2302.12692, 2023.

[18] H. Huang and O. Zheng, "ChatGPT for shaping the future of dentistry: the potential of multi-modal large language model," Int. J. Oral Sci., vol. 15, no. 1, p. 29, July 2023, doi: 10.1038/s41368-023-00239-y.

[19] V. Sorin, E. Klang, M. Sklair-Levy, and I. Cohen, "Large language model as a support tool for breast tumor board," npj Breast Cancer, vol. 9, no. 1, p. 44, May 2023, doi: 10.1038/s41523-023-00557-8.

[20] A. Russell-Gilbert et al., "AAD-LLM: adaptive anomaly detection using large language models," in Proc. IEEE Int. Conf. Big Data, Washington, DC, USA, pp. 4194–4203, Dec. 2024, doi: 10.1109/BigData62323.2024.10825679.

[21] K. P. Sinaga and M.-S. Yang, "Unsupervised K-Means clustering algorithm," IEEE Access, vol. 8, pp. 80716–80727, 2020, doi: 10.1109/ACCESS.2020.2988796.

[22] X. Li, Z. Xu, H. Peng, H. Wang and Q. Huang, "Analysis and application of clustering validity indexes," 2022 International Conference on Machine Learning and Intelligent Systems Engineering (MLISE), Guangzhou, China, 2022, pp. 131-136, doi: 10.1109/MLISE57402.2022.00034.

[23] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, New Orleans, LA, USA, 2022, Art. no. 1800.